Name: Aayush Mansukh

Topic: Network/ZTNA by Nile

NetSentinel

1. **Product Name**: NetSentinel

Why it is best ?

• Implements Zero Trust Security

• Prevents Unauthorized Access

• Combines Access Control with Monitoring

• Practical and Scalable Solution

NetSentinel is a Zero Trust-based network security system that verifies every access request and continuously monitors user activity to ensure secure communication.

2. **Core Idea of NetSentinel**

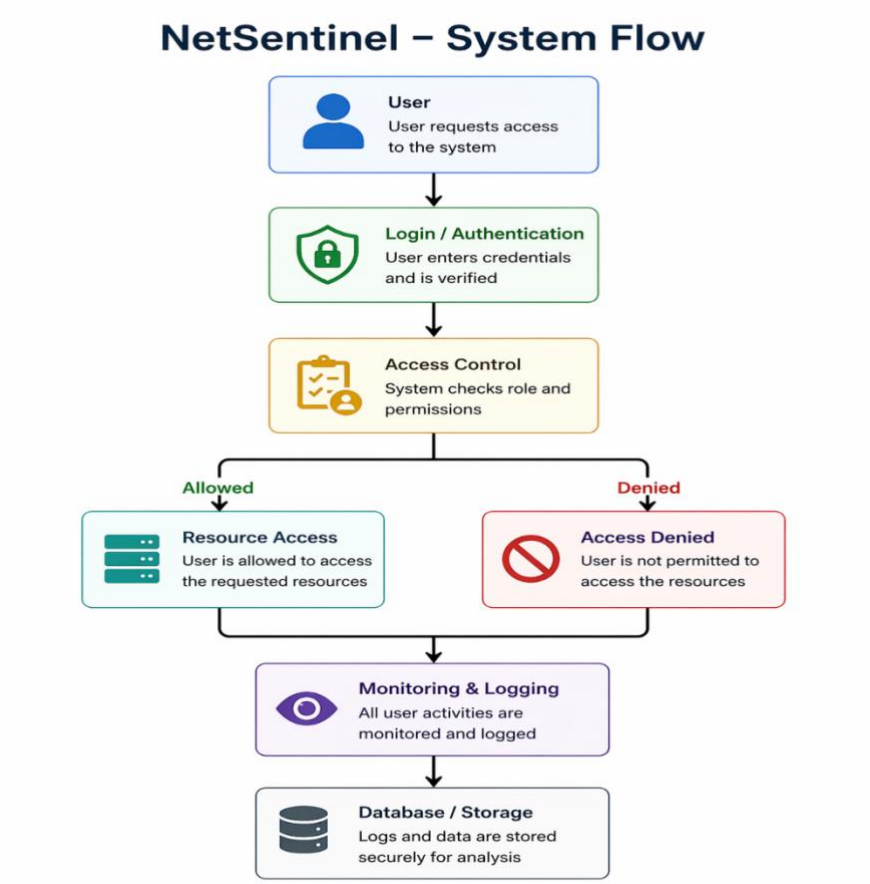
NetSentinel is a Zero Trust-based system that verifies every access request and continuously monitors user activity to ensure secure network communication.

NetSentinel is designed to address the limitations of traditional network security systems that trust users after login. Instead of relying on one-time authentication, it follows a Zero Trust approach where every user and device must be verified before accessing any resource.

The system checks user identity, applies role-based access control, and continuously monitors user activity to detect any unusual or unauthorized behavior. All actions are logged to ensure transparency and traceability.

By combining authentication, access control, and monitoring into a single system, NetSentinel ensures that network access remains secure, controlled, and continuously verified, reducing the risk of internal and external threats.

3. **System Architecture & Flow**



This workflow illustrates how NetSentinel verifies user identity, controls access, and continuously monitors activity to ensure secure network communication.

I have also included a Mermaid diagram link which provides an interactive view of the system workflow for better understanding.

https://mermaid.ai/d/2e12fe87-ef1e-4dd3-8e07-d48ea897781e

4. **File System Structure of NetSentinel**

```
netsentinel/
│
├── backend/                # Handles core system logic and processing
│   ├── api/                # Contains API routes for handling requests
│   │   ├── auth_routes.py   # Manages login and authentication requests
│   │   ├── access_routes.py # Handles access control operations
│   │
│   ├── services/                  # Core engines of the system
│   │   ├── auth_engine.py           # Verifies user identity
│   │   ├── access_control_engine.py  # Decides allow or deny access
│   │   ├── risk_detector.py         # Detects suspicious activity
│   │   ├── monitoring_engine.py      # Tracks and logs user actions
│   │
│   ├── models/             # Database structure definitions
│   │   ├── user_model.py    # Stores user information
│   │   ├── log_model.py     # Stores activity logs
│   │
│   └── main.py             # Entry point of backend server
│
```

```
├── frontend/              # User interface of the system
│   ├── login.html         # Login page for users
│   ├── dashboard.html     # Displays system data and status
│   ├── styles.css         # Styling for UI components
│
├── database/              # Data storage layer
│   └── db.sqlite          # Stores users and logs
│
└── config/                # Configuration settings
    └── settings.py        # Stores system configuration values
```

The file system is structured into backend, frontend, and database. The backend contains core engines like authentication and access control. The frontend handles user interaction, and the database stores user data and logs.

What This File System Will Do?

◈ Backend

Handles the core logic of NetSentinel, including user authentication, access control, risk detection, and monitoring of network activity.
This acts as the **main processing unit** of the system.

◈ Frontend

Provides the user interface where users can log in and interact with the system. Displays access status, logs, and system responses.

◈ Services (Core Engines)

Contains the main engines such as authentication, access control, risk detection, and monitoring.
These modules ensure secure and controlled access to network resources.

◈ Database

Stores important data such as user information, login records, and activity logs.
Ensures data is securely maintained and easily accessible.

◈ Config

Stores system settings and configuration values required for smooth functioning of the application.

5. **Dependencies & Technology Stack**

◈ Backend

- Python (Flask) → Handles server-side logic and request processing

◈ Frontend

- HTML, CSS → Builds user interface
- JavaScript → Handles user interactions

◈ Database

- SQLite → Stores user data and activity logs
- SQL → Used for querying and managing data

◈ Tools

- Git → Version control and project management
- Wireshark → Used for understanding network traffic (conceptual use)

6. **Working Mechanism & Triggers of NetSentinel**

NetSentinel verifies every user request, controls access, and continuously monitors activity using a Zero Trust approach.

◈ **Step 1: User Login**

User enters credentials through the login interface.

◈ **Step 2: Authentication**

System verifies user identity using stored data.

◈ **Step 3: Access Control**

User permissions are checked based on role (admin/user).

◈ **Step 4: Decision Making**

System decides whether to **allow or deny** access.

◈ **Step 5: Resource Access**

If allowed, user can access the requested resource.

◈ **Step 6: Monitoring & Logging**

All user activities are tracked and stored for security analysis.

TRIGGERS

◈ Login Trigger

Activated when user attempts to log in.

◈ Access Request Trigger

Activated when user tries to access a resource.

◈ Unauthorized Access Trigger

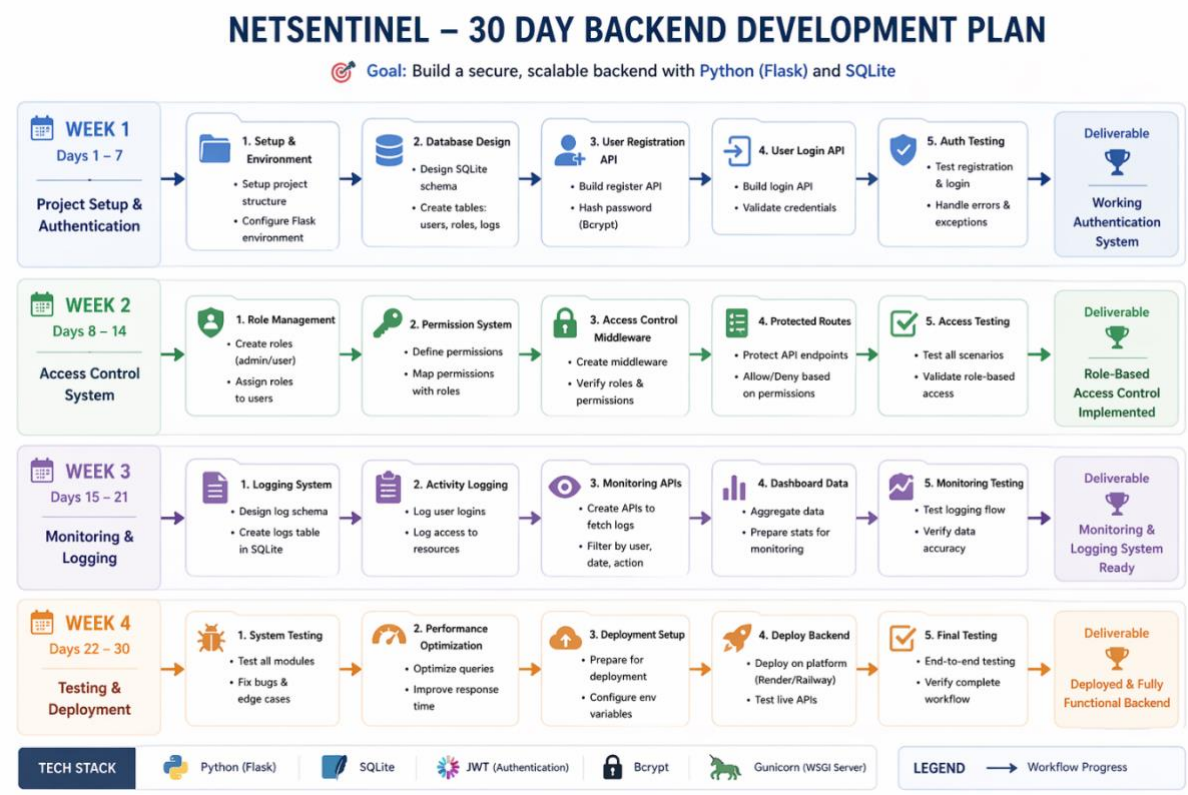
Activated when invalid or restricted access is attempted.

◈ Monitoring Trigger

Activated to continuously track and log user activity.

7. 30-Day Development Roadmap

The development plan focuses on building a functional backend first, followed by integration, testing, and deployment.



The workflow shows a 4-week plan covering authentication, access control, monitoring, and deployment to build a complete backend system.

8. **What I need to build this project**

- GitHub → for code management

- Render / Railway → for deployment
- Basic system resources (laptop, internet)
- Open-source tools for development

9. Technologies Used

- Backend: Python (Flask)
- Frontend: HTML, CSS, JavaScript
- Database: SQLite
- Version Control: Git

WHY?

I chose Flask because it is lightweight and easy to use. HTML/CSS for simple UI, and SQLite for easy database management. These technologies are simple, efficient, and suitable for building a scalable and working prototype.

10. Market Research & Product Differentiation

◈ Existing Solutions

- Traditional systems trust users after login
- Limited monitoring of user activity
- Complex and expensive security solutions

◈ Limitations

- No continuous verification
- Lack of real-time monitoring
- High implementation complexity

◈ NetSentinel Advantage

- Follows Zero Trust approach
- Verifies every access request
- Simple and cost-effective solution
- Designed as an easy-to-implement prototype

NetSentinel differentiates itself by combining security, simplicity, and real-time monitoring in a single system.